



تمرین عملی دوم (کوبرنتیز)

مبانی رایانش ابری - دکتر ثباتی مقدم - بهار ۱۴۰۵

شما عضو تیم DevOps یک شرکت نرم‌افزاری هستید. این شرکت یک اپلیکیشن دارد که قبلاً با Docker Compose در قالب تمرین عملی اول پیاده‌سازی کردید. حالا قرار است این سیستم به محیط Kubernetes منتقل شود تا برای استقرار در production آماده باشد.

تیم توسعه انتظار دارد این مهاجرت با در نظر گرفتن **امنیت، پایداری، مقیاس‌پذیری** و قابلیت مانیتورینگ انجام شود. همچنین، انتظار می‌رود محیطی طراحی شود که به راحتی قابل نگهداری و ارتقا باشد.

شما به عنوان مسئول این مهاجرت، باید اپلیکیشن را با تمام اجزای آن در Kubernetes پیاده‌سازی و تنظیم کنید.

مرحله اول: آماده‌سازی زیرساخت

ابتدا باید یک کلاستر محلی Kubernetes ایجاد کنید. می‌توانید از ابزارهایی مانند Kind یا Minikube استفاده کنید. پس از راه‌اندازی، با اجرای دستوراتی مانند `kubectl get nodes` و `kubectl cluster-info` مطمئن شوید که کلاستر شما آماده است. یک namespace جدید به نام `user-system` تعریف کنید تا پروژه شما از سایر سرویس‌های کلاستر جدا باشد.

مرحله دوم: راه‌اندازی پایگاه داده

پایگاه داده سیستم دقیقاً همان پایگاه داده ای است که در تمرین عملی اول پیاده‌سازی کرده اید. ابتدا باید اطلاعات حساس مانند `username` و `password` دیتابیس را در قالب Secret ذخیره کنید. سپس یک StatefulSet تعریف کنید که یک Pod دیتابیس را به همراه یک PersistentVolumeClaim راه‌اندازی کند تا داده‌ها در صورت ری‌استارت یا جابه‌جایی Pod از بین نروند.

مرحله سوم: راه اندازی Backend اپلیکیشن

سرویس بک اندی که در تمرین عملی اول پیاده سازی کرده اید، باید به صورت یک Deployment اجرا شود. اطلاعات حساس مثلا کلید JWT را در Secret ذخیره کرده و تنظیمات عمومی (مانند آدرس دیتابیس، پورت ها و تنظیمات قابل تغییر) را در ConfigMap قرار دهید. هنگام تعریف Deployment، حداقل دو Replica در نظر بگیرید و برای هر کدام از Pod ها، liveness و readiness probe تعریف کنید تا وضعیت سلامتی آن ها بررسی شود.

برای اطمینان از اینکه backend فقط زمانی اجرا شود که دیتابیس آماده است، از یک InitContainer استفاده کنید که قبل از اجرای اپلیکیشن، اتصال به پایگاه داده را بررسی کند.

مرحله چهارم: راه اندازی وب سرور

در این مرحله شما باید با استفاده از Nginx (یا هر وب سرور دیگری که در تمرین عملی اول استفاده کرده اید)، یک ورودی برای دسترسی به backend ایجاد کنید. Nginx درخواست هایی که از کاربر نهایی می رسد را به backend هدایت می کند. یک Deployment برای Nginx ایجاد کرده و تنظیمات آن را با استفاده از ConfigMap ارائه دهید. این Nginx می تواند به صورت Service از نوع NodePort در دسترس باشد، یا اگر Ingress Controller نصب کردید، از Ingress استفاده کنید.

- استفاده از هر وب سرور دیگری غیر از Nginx مجاز می باشد و مشکلی نخواهد بود.

مرحله پنجم: بررسی قابلیت اطمینان

قابلیت هایی مثل HPA افزایش خودکار تعداد Replica های PodDisruptionBudget، backend (برای جلوگیری از حذف همزمان چند Pod)، و Affinity (اجرای Pod ها در Node های متفاوت) را نیز پیاده سازی کنید. همچنین می توانید با ابزارهایی مثل K9s وضعیت Pod ها را بررسی کنید.

- انجام یکی از موارد گفته شده الزامی و دو مورد دیگر اختیاری می باشد. (HPA, PDB, Affinity)

مرحله ششم: Redis برای کش (اختیاری)

برای بهبود کارایی سیستم، قرار است از Redis برای ذخیره‌سازی اطلاعات کش‌شده استفاده شود. Redis را در قالب یک Deployment ساده اجرا کنید و یک Service از نوع ClusterIP برایش تعریف نمایید. مانند مرحله قبل، دسترسی به Redis را با استفاده از NetworkPolicy فقط به backend محدود کنید.

سپس یک Service از نوع ClusterIP برای دیتابیس بسازید تا سایر سرویس‌ها بتوانند از طریق DNS به آن متصل شوند. در این مرحله، همچنین باید یک NetworkPolicy بنویسید که فقط به backend اجازه اتصال به دیتابیس را بدهد، نه سایر Podها.

مرحله هفتم: تعریف مسیرهای ورودی و امنیت آن (اختیاری)

از Ingress استفاده کنید، باید یک Ingress Controller مثل NGINX نصب کرده و سپس یک Ingress Resource بسازید که مسیرهای مختلف مثل /api/ را به backend هدایت کند. در همین مرحله می‌توانید تنظیماتی مانند محدود کردن دسترسی از IP های خاص یا کنترل تعداد درخواست‌ها را با استفاده از annotation های امنیتی پیاده‌سازی کنید.

مرحله هشتم: مانیتورینگ و مشاهده‌پذیری (اختیاری)

در یک محیط واقعی، امکان مشاهده وضعیت سیستم بسیار مهم است. شما باید Prometheus را برای جمع‌آوری اطلاعات عملکردی (مثل مصرف CPU، memory، وضعیت health) نصب کنید. اگر می‌خواهید گزارش‌ها را به صورت گرافیکی ببینید، Grafana را هم نصب کرده و به Prometheus وصل کنید.

تنظیمات Prometheus باید به شکلی باشد که بتواند متریک‌های backend و دیتابیس را جمع‌آوری کند. در Grafana یک داشبورد تعریف کنید که اطلاعات مهم کلاستر (مثل وضعیت Podها، مصرف منابع، خطاها و ...) را نمایش دهد. برای امنیت بیشتر، یوزرنیم و پسورد داشبورد را در Secret ذخیره و از طریق Deployment استفاده کنید.

نکات مهم برای تحویل:

- تمام فایل‌های YAML (در پوشه‌های جدا مثل db/, monitoring/, backend/, ...) •
- یک فایل README.md با توضیح مراحل انجام‌شده، **مشکلاتی که برخورد کردید**، و توضیح دلیل اینکه **چطور مشکل را برطرف کردید**. نحوه اتصال سرویس‌ها و نحوه تست آن‌ها •
- چند اسکرین شات به طور دلخواه از سلامت و درست کار کردن اپلیکیشن و کلاستر •
- همه فایل‌ها را در قالب یک فایل zip آپلود کنید. •
- در صورت اینکه تمایل داشتید هر کدام از بخش‌های اختیاری را انجام دهید قبل از انجام با حل تمرین مربوطه هماهنگ کنید، زیرا لزوماً شامل نمره اضافه نخواهد بود. •
- توجه کنید **تسلط** روی تمرین بسیار مهم است و در نهایت نمره کل شما در تسلط **ضرب** خواهد شد، زمان ارائه تمرین هم بعد از مهلت ارسال متعاقباً اعلام خواهد شد. •
- انجام پروژه در گروه‌های **۲ نفره** قابل انجام می‌باشد که هر دو نفر باید به تمامی مراحل انجام پروژه مسلط باشند. •

موفق باشید.